

NSDSYST Machine Project Specification

Distributed "Mini-Splunk" Log Analytics Ecosystem

Project Overview

This machine project bridges theory and practice by requiring students to design and implement a resilient, scalable, and decentralized log analytics system capable of handling the complexities of a modern computing environment. Shifting away from centralized processing, this ecosystem will ingest, parse, and analyze standard syslog files across a network of fully decoupled components.

The system will utilize robust inter-process communication to handle bulk log file uploads and must withstand live system stress testing with autonomous failure recovery, demonstrating a deep understanding of resource coordination, state management, and data consistency.

Learning Outcomes Addressed

- **CLO1:** Analyze and explain the fundamental architectural patterns, communication models, and infrastructure requirements necessary to design scalable distributed systems.
- **CLO2:** Design and implement a functional distributed system utilizing modern middleware, containerization, and inter-process communication techniques.
- **CLO3:** Evaluate and resolve challenges in distributed environments related to task synchronization, resource coordination, and data consistency to ensure system reliability and security.

Distributed Architecture & Core Components

The ecosystem must abandon tightly coupled monolithic traits and implement the following decoupled infrastructure:

1. **The Forwarder (CLI Client):** Reads local syslog files and transmits the raw payload over the network rather than streaming directly to a centralized processing server.
2. **Communication Middleware / Inter-Process Communication (IPC) Layer:** A robust communication mechanism (RPC, REST, or Message Queues) that connects components and facilitates the transfer of log payloads and search queries. It must effectively manage workload distribution between the gateway and worker nodes and minimize synchronous blocking.
3. **Distributed Worker Nodes:** Multiple containerized worker processes that receive log data via the chosen IPC layer. These nodes must parse the syslog entries into standard components (Timestamp, Hostname, Process, Severity, Message) using regular expressions.
4. **Decoupled & Distributed Data Layer (WITH RESTRICTIONS):** Parsed results must be seamlessly and accurately aggregated into a distributed storage engine.
 - *Limitation:* The use of a single, centralized database instance (e.g., a standalone MySQL or PostgreSQL container) is **strictly prohibited**. A centralized database acts as a single point of failure and a system bottleneck, violating the core principles of the course.

- *Requirement:* Students must implement a partitioned distributed database (e.g., clustered MongoDB, Cassandra, Elasticsearch) or manually shard a relational database across multiple independent nodes to ensure high availability and distributed querying.
5. **API Gateway / Search Head:** A service discovery or gateway node that receives client interrogation queries, orchestrates scatter-gather searches across the data layer shards, and aggregates the results to return to the client.

Functional Requirements (CLI Commands)

The client interface must support the following operations, adapted for distributed execution:

- **INGEST <file_path> <Gateway_IP>:** Streams a local file to the system for distributed asynchronous processing.
- **QUERY <Gateway_IP>: SEARCH_DATE "<date_string>":** Returns all parsed log lines matching a specific date across the distributed data layer.
- **QUERY <Gateway_IP>: SEARCH_HOST <hostname>:** Returns all parsed log lines originating from a specific machine.
- **QUERY <Gateway_IP>: SEARCH_DAEMON <daemon_name>:** Filters indexed logs to show only those generated by a specific service.
- **QUERY <Gateway_IP>: SEARCH_SEVERITY <severity_level>:** Filters logs matching a specific severity level.
- **QUERY <Gateway_IP>: SEARCH_KEYWORD <keyword>:** Scans the distributed logs and returns the full log lines containing the exact keyword.
- **QUERY <Gateway_IP>: COUNT_KEYWORD <keyword>:** Returns an aggregate count of how many times the keyword appears across all shards/nodes.
- **PURGE <Gateway_IP>:** Deletes all indexed log entries. The system must utilize **distributed coordination mechanisms (e.g., distributed locks)** to ensure no other worker nodes are currently reading or writing to the database shards while the clear operation is in progress.

Technical Specifications & Fault Tolerance

- **Implementation Strategy:** Modern middleware, REST APIs, RPC, or message queues must be utilized.
- **Strict Containerization & Deployment:** Building and deploying the system must rely entirely on containerization (e.g., Docker). The entire ecosystem—including the chosen IPC middleware and the distributed Data Layer nodes—must be orchestrated using a unified manifest (e.g., docker-compose.yml) to ensure a seamless, "1-click" automated deployment.
- **Fault Tolerance (Chaos Testing):** The architecture must feature flawless state management. The system will be subjected to chaos/concurrency tests (e.g., forcefully killing a worker node mid-ingestion) and must demonstrate zero data loss, zero duplicate processing, and full autonomous recovery.

Deliverables & Grading

This project fulfills the Major Course Output (MCO) requirements and is worth 40% of the final grade. It will be evaluated out of 100 points based on the course rubric.

- **Project Architecture Diagram (Due Week 8):** A professional diagram explaining fault tolerance, the chosen IPC model, container orchestration, and how the data layer is partitioned/sharded.
- **Machine Project Source Code (Due Week 13):** A complete system demonstrating workload distribution and task synchronization. Submission must include all source code alongside the required container build files (Dockerfiles) and orchestration manifests (e.g., docker-compose.yml).
- **Demonstration & Documentation (Due Week 14):** A technical walkthrough and defense of the system architecture under stress testing. The final written documentation must strictly follow the **IEEE conference template format, must not exceed a maximum of 6 pages**, and must be structured according to the following outline:
 - **I. Introduction:** Background, motivation for a decoupled distributed architecture, and project objectives aligned with course CLOs.
 - **II. Algorithms and Implementation:**
 - *A. System Architecture:* Detailed explanation of the components and the required professional architecture diagram.
 - *B. Inter-Process Communication (IPC):* Explanation of the chosen middleware, workload distribution, and network handling.
 - *C. State Management and Consistency:* Documentation of distributed locks and data layer partitioning.
 - *D. Deployment and Containerization:* Explanation of the Docker orchestration strategy and unified manifest logic.
 - **III. Test and Result:**
 - *A. Core Functionality and Performance:* Results from log ingestion and distributed scatter-gather queries.
 - *B. Fault Tolerance and Chaos Testing:* Documented proof from rigorous stress testing demonstrating zero data loss and autonomous recovery when nodes fail.
 - **IV. Conclusion:** Summary of achievements, resolution of distributed computing challenges, and potential future architectural improvements.
 - **References:** Properly cited sources (e.g., textbooks, API docs) following the IEEE format.

Course Policies & AI Usage

- **Intellectual Honesty:** Any requirements submitted for grading should reflect group work effort and intellectual honesty. Anybody caught cheating or violating intellectual honesty will be subject to appropriate disciplinary action.
- **Generative AI Use Policy:** The use of generative AI to build the core system architecture or write the primary business logic from scratch is strictly prohibited. Limited use of generative AI is permitted strictly for logic checking, syntax checking, and error debugging of student-produced code, provided that students explicitly disclose such use. Failure to properly disclose permitted use, or using it beyond the defined scope, constitutes academic dishonesty.

AI Usage Declaration: The text of this project specification was refined with the assistance of generative AI. Its usage was strictly limited to adjusting grammar, enhancing sentence structure, and balancing the technical limitations of the project. The academic design and evaluation criteria remain the original work of the instructor.